

Software System Report Vision-Based Robotic Sorting Arm

1. مقدمة (Introduction)

المشروع عبارة عن نظام ذكي لذراع روبوتية تقوم بعملية **Object Sorting** بشكل آلي باستخدام تقنيات :

- **Computer Vision**
- **Artificial Intelligence**
- **Embedded Systems**
- **Robotics**

يعتمد النظام على تقسيم واضح بين:

- **Software (Python (على اللابتوب**
- **Embedded System (ESP32)**

حيث يتم تحليل البيئة واتخاذ القرار على اللابتوب، بينما يتم تنفيذ الحركة على الذراع الروبوتية.

2. معمارية النظام (System Architecture)

يتكون النظام من ثلاث طبقات رئيسية :

1. Vision & AI Layer (Python)

- معالجة الصور
- اكتشاف الأجسام
- اتخاذ القرار

2. Communication Layer

- إرسال الأوامر للروبوت (Wired أو Wireless)

3. Control Layer (ESP32)

- تنفيذ الحركة

3. مكونات النظام البرمجية (Software Components)

أولاً: بيئة Python (Main Software)

البرنامج الرئيسي مكتوب بلغة **Python** ويعمل على اللابتوب، وهو المسؤول عن:

- تشغيل الكاميرا (USB Camera)
- معالجة الصور
- اكتشاف الأجسام
- اتخاذ القرار
- إرسال أوامر الحركة للروبوت

المكتبات المستخدمة:

1. مكتبة OpenCV

اسم المكتبة:

cv2

وظيفتها:

- التعامل مع الكاميرا
- معالجة الصور
- اكتشاف الأجسام
- تحليل الألوان

أهم الدوال المستخدمة:

cv2.VideoCapture(0)

→ تشغيل الكاميرا

cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

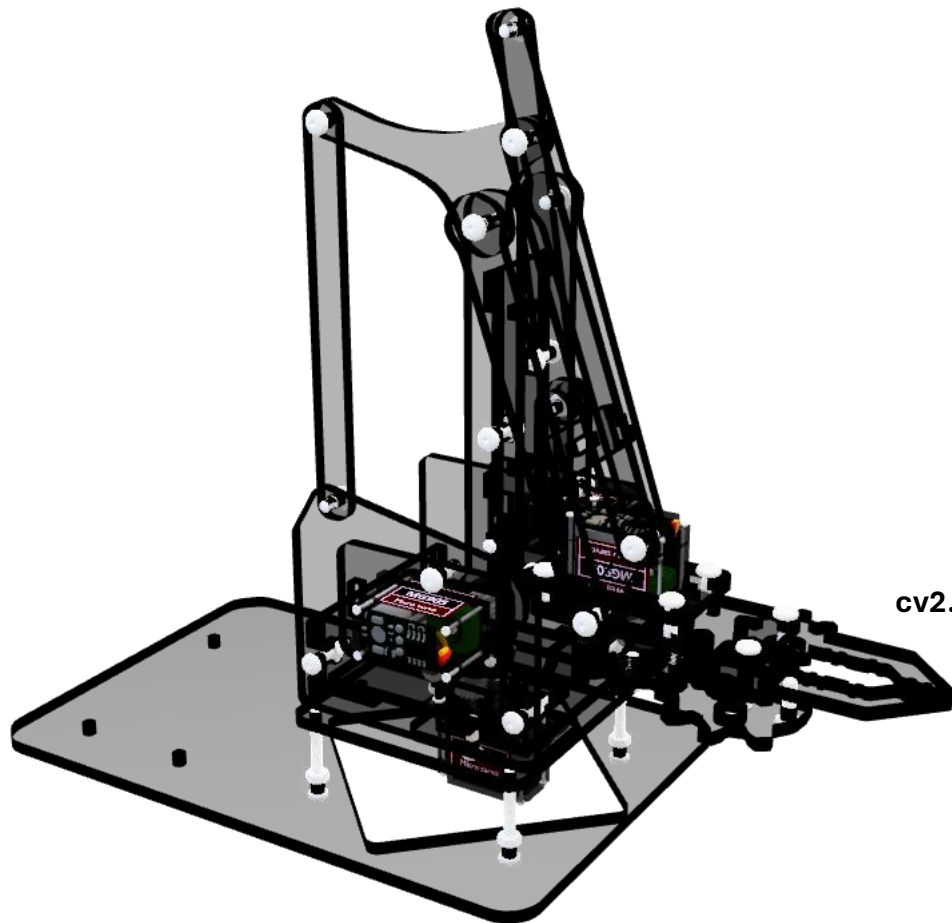
→ تحويل الصورة إلى HSV

cv2.inRange(hsv, lower, upper)

→ عزل لون معين

cv2.findContours()

→ اكتشاف حدود الأجسام



2. مكتبة NumPy

numpy

وظائفها:

- العمليات الحسابية
 - التعامل مع مصفوفات الصور
- مثال:

`np.array()`

3. مكتبة Requests

requests

وظائفها:

إرسال أوامر للروبوت عبر الشبكة (HTTP)

مثال:

`requests.get(url)`

4. مكتبة PyQt

PyQt

وظائفها:

- إنشاء واجهة المستخدم
- عرض الكاميرا
- عرض حالة النظام



4. وحدة الرؤية (Vision System)

الكاميرا المستخدمة

في النسخة الحالية من المشروع:

USB Camera متصلة باللابتوب

السبب:

- سهولة الاستخدام (Plug & Play)
- توافق مباشر مع OpenCV
- تقليل التعقيد في الهاردوير

ملاحظة مهمة

هذه النسخة تعتبر:

(نموذج مصغر) Prototype

بينما في النسخ المتقدمة يمكن استخدام :

• Raspberry Pi

• Jetson Nano

مع كاميرات احترافية مثل:

CSI Camera (IMX219)

الفكرة هنا :

نحن نحاكي النظام الصناعي، لكن باستخدام Setup أبسط مناسب للإمكانيات.

5. معالجة الصورة (Image Processing Pipeline)

تمر الصورة بالمراحل التالية:

1. التقاط الصورة

`ret, frame = cap.read()`

2. تحويل الألوان

`hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`

3. عزل اللون

`mask = cv2.inRange(hsv, lower, upper)`

4. إزالة الضوضاء
`cv2.morphologyEx()`

5. اكتشاف الأجسام
`cv2.findContours()`

6. اختيار الجسم الأكبر
`max(contours, key=cv2.contourArea)`

7. حساب مركز الجسم
 $cx = M["m10"] / M["m00"]$
 $cy = M["m01"] / M["m00"]$

6. حساب المسافة (Distance Estimation)

Ultrasonic Sensor

- مثبت أسفل الـ Gripper
- بزاوية : 15° Forward Tilt
- الهدف:
- تجنب قراءة الأرض مباشرة
- تحسين دقة القياس

المعادلة المستخدمة

$Distance = (Time \times Speed\ of\ Sound) / 2$

في كود ESP :

`distance = duration * 0.034 / 2;`

7. تحويل الإحداثيات (Coordinate Mapping)

Image Coordinates $\rightarrow$ Robot Coordinates

المعادلة:

$X_{robot} = (cx / width) \times workspace_width$

8. تخطيط الحركة (Motion Planning)

يتم استخدام:

Inverse Kinematics

لحساب زوايا:

- Base
- Shoulder
- Elbow

9. التواصل بين اللابتوب و(ESP32 (Communication System

النظام يدعم طريقتين اتصال:

الطريقة الأولى (WiFi) Wireless :

الفكرة:

- ESP32 يعمل كـ:
- Access Point (Hotspot)
- اللابتوب يتصل به

إرسال الأوامر:

HTTP Request

مثال:

`/move?base=90&shoulder=45&elbow=30&gripper=80`



المميزات:

- بدون أسلاك
- مرونة عالية
- Real-Time تقريبًا

الطريقة الثانية (USB Serial) : Wired

باستخدام مكتبة:

serial (PySerial)

مثال:

```
ser.write("MOVE 90 45 30 80\n")
```

المميزات:

- ثابت جدًا
- لا يتأثر بالشبكة
- مناسب كـ Backup

نقطة مهمة

النظام مصمم بحيث:

Dual Communication System

أي:

- الوضع الأساسي = Wireless
 - (Fail-safe) نظام احتياطي = Wired
- وهذا يزيد **Reliability** بشكل كبير.

10. وحدة التحكم (ESP32 Control System)

المكتبات المستخدمة

Servo.h

WiFi.h

WebServer.h

المهام

- استقبال الأوامر
- تحليل البيانات
- تحريك السيرفو
- قراءة الحساسات

11. التحكم في السيرفو

servo.write(angle);

نطاق:

0° → 180°

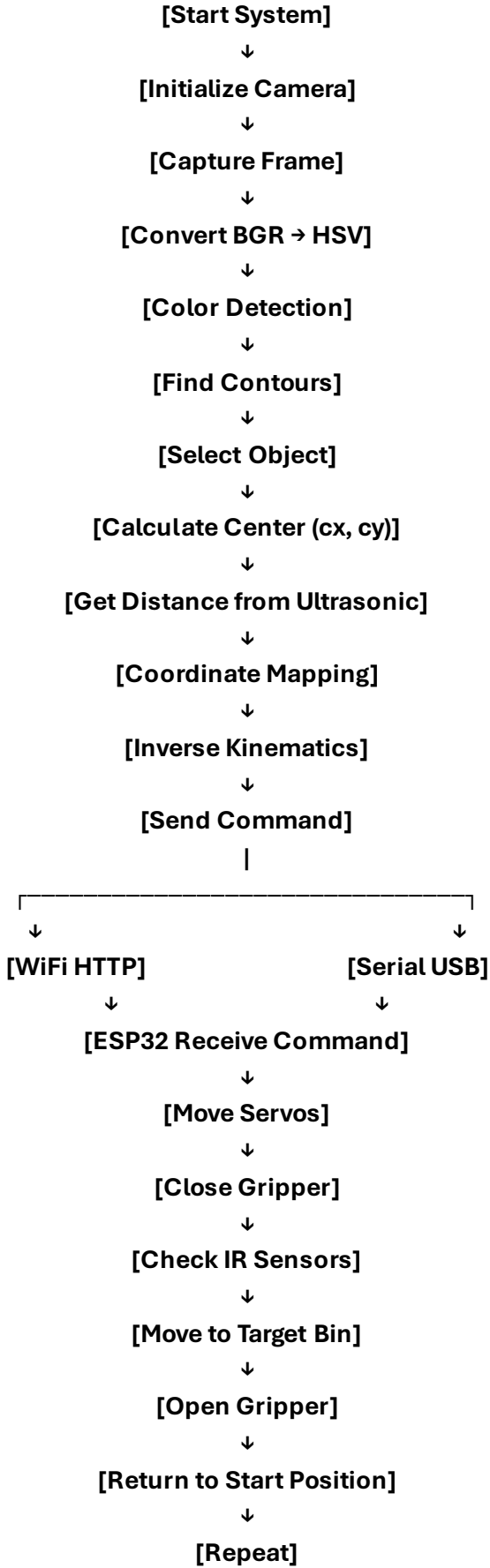
12. التحكم في القبضة (Gripper)

- فتح
- غلق

تأكيد الإمساك باستخدام IR Sensor

- يوجد IR 2 داخل القبضة
- إذا قرأوا HIGH الإمساك ناجح

Flowchart (System Workflow)



13. دورة عمل النظام (Full Workflow)

1. التقاط الصورة
2. اكتشاف الجسم
3. حساب الموقع
4. قياس المسافة
5. حساب الحركة
6. إرسال الأوامر
7. تحريك الذراع
8. الإمساك
9. التأكد بـ IR
10. نقل للصندوق
11. الإفلات
12. العودة

14. مميزات النظام البرمجي

- Modular Architecture
- قابل للتطوير
- يدعم AI مستقبلاً
- Dual Communication
- Real-Time Processing

15. الخلاصة

النظام يمثل نموذجاً متكاملًا يجمع بين:

Computer Vision + AI + Embedded Systems + Robotics

مع استخدام:

USB Camera (Prototype)

كنموذج أولي، مع إمكانية التطوير إلى:

Jetson Nano / Raspberry Pi + CSI Camera

لتحقيق أداء صناعي متقدم.

